



Analysis & Development of Algorithms

III

Anton Kovantsev

https://t.me/pop_antonij

Terms for first order methods

Problem: $f: \mathbb{R}^n \rightarrow \mathbb{R}$ is convex; $f = f(x)$, where $x = (x_1, x_2 \cdots x_n)$ is a row-vector. To solve the optimization problem $f(x) \rightarrow \min_{x \in Q}$ means to find $x^* \in Q$, where Q is the region of acceptability, such that f reaches a minimal value at x^* . $x^* = \operatorname{argmin}_{x \in Q} f(x)$.

Remark: The approximation error is $\varepsilon > 0$. In the iterative algorithms below, we stop if $\|a_n - a_{n-1}\| < \varepsilon$, supposing, say, $x^* \approx (a_n + a_{n-1})/2$ with error ε .

Recall: The gradient is a multi-variable generalization of the derivative The gradient of a differentiable function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ at a is the n -dimensional column-vector $\nabla f(a)$ whose elements are

$f(x)$	$f'(x)$	$f''(x)$
x	1	0
x^3	$3x^2$	$6x$
$\sin x$	$\cos x$	$-\sin x$
$\ln x \mid \log_n x$	$1/x \mid 1/(x \ln n)$ $x > 0$	$1/x^2 \mid 1/x^2 \ln n$
$ x $	$-1 \ x < 0$ $1 \ x > 0$ $undefined \ x = 0$	$0 \ x \neq 0$ $undefined \ x = 0$

$$\nabla f(a) = \left. \frac{\partial f}{\partial x_i} \right|_a \quad i=1, \dots, n$$

Example: $f(x) = 2x_1 + 3x_2^2$
 $\nabla f(x) = (2, 6x_2)^T$
 If $a = (0, 1)$, then $\nabla f(a) = (2, 6)^T$

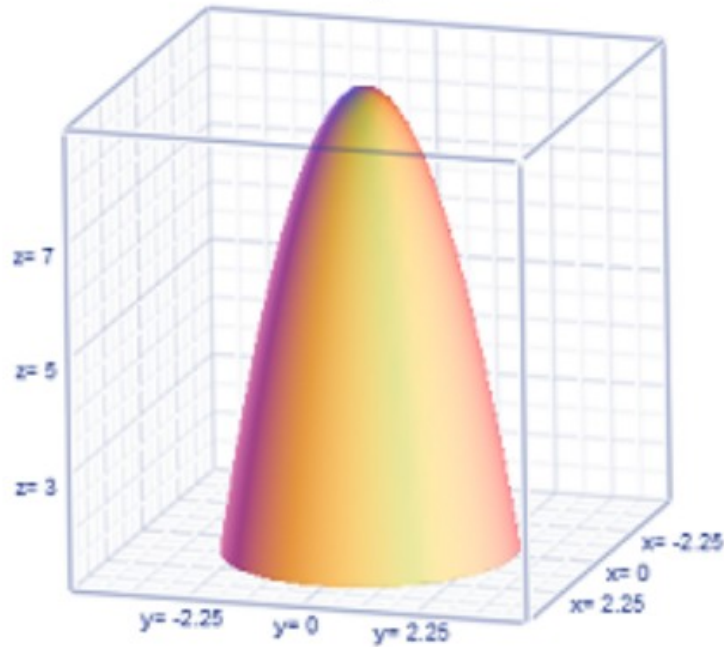
If the gradient of our function is not the zero vector at a , it has the direction of fastest increase of the function at a .

Gradient

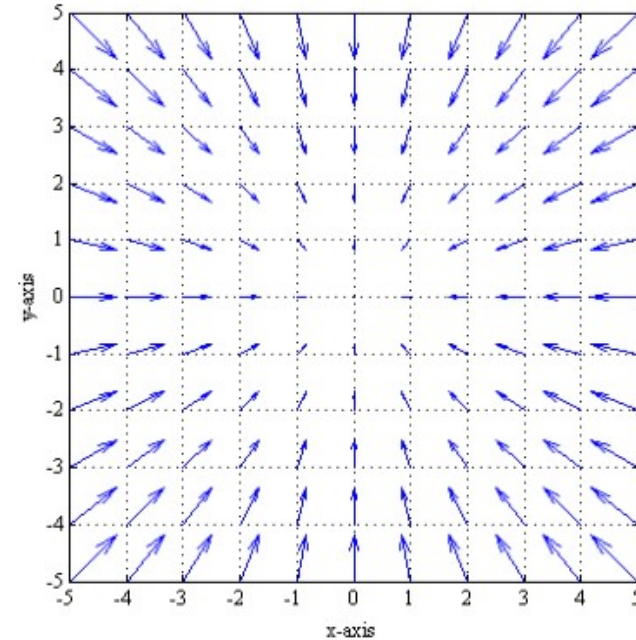
$$z = f(x, y) = 9 - (x^2 + y^2)$$

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{pmatrix} = \begin{pmatrix} -2x \\ -2y \end{pmatrix}$$

3D plot



Gradient field



Hessian

The Hessian matrix, or Hessian, is a square matrix of second-order partial derivatives that describes the local curvature and is the generalization of the second derivative for multi-variable functions.

The Hessian of a twice differentiable function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ at $\mathbf{a}$ is the $n \times n$ matrix $\mathbf{H}f(\mathbf{a})$ whose elements are

$$H_{i,j} = \left. \frac{\partial^2 f}{\partial x_i \partial x_j} \right|_{\mathbf{a}} \quad i, j = 1, \dots, n$$

Example:

$$f(x_1, x_2) = x_1^2 + 3x_1x_2^3 \quad \mathbf{a} = (1, 2)$$

$$H_{1,1} = \frac{\partial^2 f}{\partial^2 x_1} = 2; H_{1,2} = \frac{\partial^2 f}{\partial^2 x_1} = 9x_2^2;$$

$$H_{2,1} = \frac{\partial^2 f}{\partial x_2 \partial x_1} = H_{1,2}; H_{2,2} = \frac{\partial^2 f}{\partial^2 x_2} = 18x_1x_2$$

$$\mathbf{H}f(\mathbf{x}) = \begin{pmatrix} \frac{\partial^2 f}{\partial^2 x_1} & \frac{\partial^2 f}{\partial x_1 \partial x_2} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial^2 x_2} \end{pmatrix} = \begin{pmatrix} 2 & 9x_2^2 \\ 9x_2^2 & 18x_1x_2 \end{pmatrix}; \quad \mathbf{H}f(\mathbf{a}) = \begin{pmatrix} 2 & 36 \\ 36 & 36 \end{pmatrix}$$

Taylor expansion

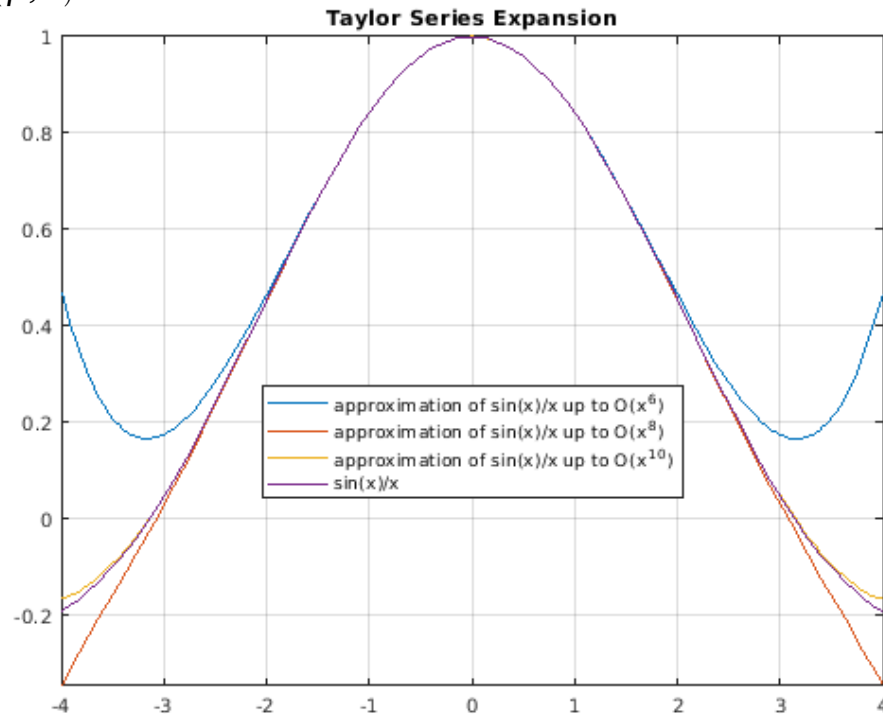
The Taylor expansion of $f: \mathbb{R} \rightarrow \mathbb{R}$ at a is

$$T_f(x) = f(a) + \frac{f'(a)}{1!}(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f'''(a)}{3!}(x-a)^3 + \dots + r_n(f, x)$$
$$r_n(f, x) = o((x-a)^n) \text{ as } x \rightarrow a$$

Generalization for $f: \mathbb{R}^n \rightarrow \mathbb{R}$ at a is

$$T_f(x) = f(a) + (x-a) \frac{\nabla f(a)}{1!} + (x-a) \frac{Hf(a)}{2!} (x-a)^T + \dots$$

x and a are defined to be row-vectors
and $\nabla f(a)$ to be a column-vector here.



Gradient descent

Gradient descent is based on the observation that if f is differentiable at a , then $f(x)$ decreases fastest in a neighbourhood of a in the direction of $-\nabla f(a)$. One may write down the following formula:

$$a_{n+1} = a_n - \beta_n \nabla f(a_n) \quad \beta_n > 0; n = 1, 2, 3, \dots; a_0 - \text{initial approximation}$$

If the factor β_n is chosen properly, then $f(a_n) \geq f(a_{n+1}) \geq f(a_{n+2}) \geq \dots$
 $a \rightarrow x^*$ as $n \rightarrow \infty$ (x^* - local minimum).

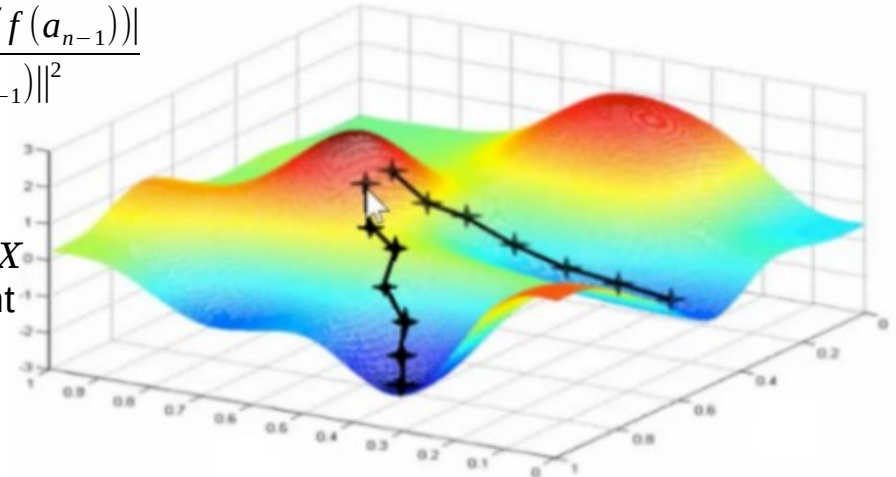
The uniform convergence is guaranteed if f is convex, ∇f is Lipschitz* and the choice of β_n is due to Barzilai-Borwein,

$$\beta_n^{BB} = \frac{|(a_n - a_{n-1})(\nabla f(a_n) - \nabla f(a_{n-1}))|}{\|\nabla f(a_n) - \nabla f(a_{n-1})\|^2}$$

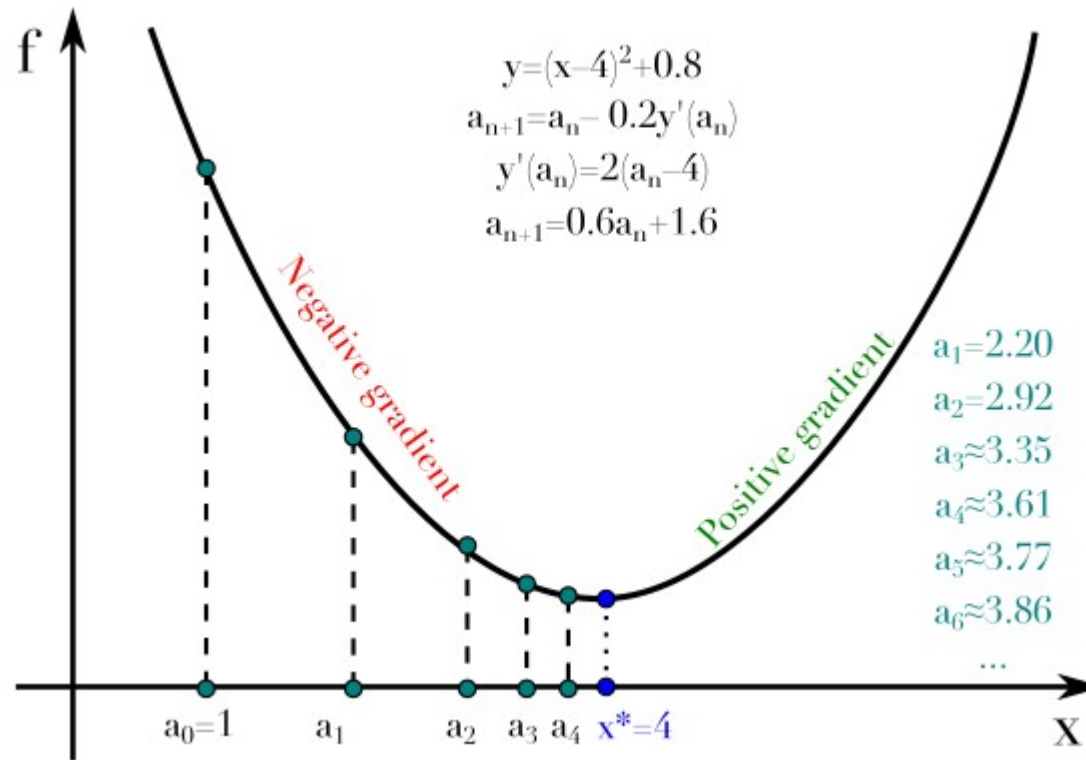
* Lipschitz continuity:

Two metric spaces (X, d_X) and (Y, d_Y) , where d_X denotes the metric on the set X and d_Y is the metric on set Y , a function $f: X \rightarrow Y$ is called Lipschitz continuous if there exists a real constant $K \geq 0$ such that, for all x_1 and x_2 in X ,

$$d_Y(f(x_1), f(x_2)) \leq K d_X(x_1, x_2)$$



Gradient descent



Nesterov's Accelerated Gradient, NAG

Nesterov acceleration modifies this by calculating the gradient not at the current position, but at a lookahead position. This subtle change gives NAG a predictive quality:

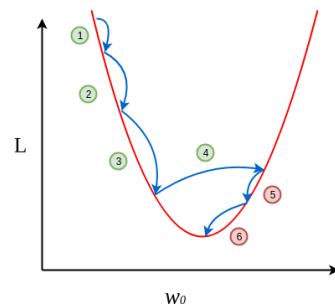
- Compute a lookahead position: $x_t + \mu v_t$;
- Evaluate the gradient at this lookahead point;
- Update the velocity and parameters accordingly.

This modification means NAG anticipates the future position, leading to better-informed updates, especially in regions where the loss function curves sharply. Compared to standard momentum, NAG tends to provide:

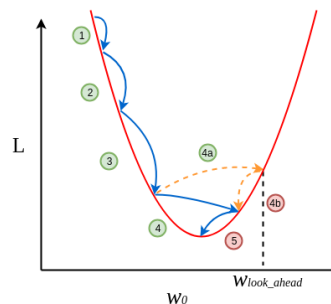
- Faster convergence in many convex and non-convex problems;
- More stable optimization paths, reducing overshooting near minima;
- Enhanced responsiveness to changes in the gradient's direction.

$$v_{t+1} = \mu v_t - \eta \nabla f(x_t + \mu v_t)$$
$$x_{t+1} = x_t + v_{t+1}$$

Nesterov's Lower Bound Theorem: No first-order method (using only function values and their gradients) can converge faster than Nesterov's accelerated gradient descent (NAG) on the class of smooth convex functions.



(a) Momentum-Based Gradient Descent



(b) Nesterov Accelerated Gradient Descent

$$\text{Green Circle} \Rightarrow \frac{\partial L}{\partial w_0} = \frac{\text{Negative}(-)}{\text{Positive}(+)}$$

$$\text{Red Circle} \Rightarrow \frac{\partial L}{\partial w_0} = \frac{\text{Negative}(-)}{\text{Negative}(-)}$$

Stochastic gradient descent

can be regarded as a stochastic approximation of gradient descent optimization, since it replaces the actual gradient (calculated from the entire data set) by an estimate thereof (calculated from a randomly selected subset of the data). For a function like:

$$Q(w) = \frac{1}{n} \sum_{i=1}^n Q_i(w)$$

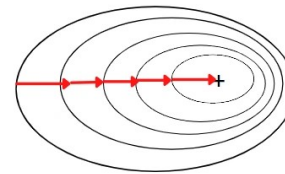
In stochastic (or "on-line") gradient descent, the true gradient of $Q(w)$ is approximated by a gradient at a single sample:

$$w := w - \eta \nabla Q_i(w)$$

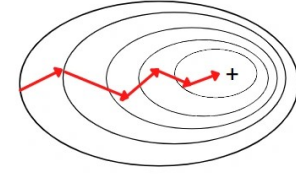
Algorithm:

- 1) Choose an initial vector of parameters w and learning rate η .
- 2) Repeat until an approximate minimum is obtained:
- 3) Randomly shuffle samples in the training set.
- 4) For $i = 1, 2, \dots, n$, do:
 $w := w - \eta \nabla Q_i(w)$.

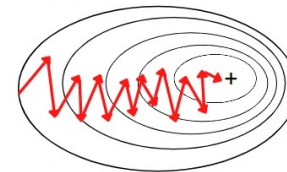
Batch Gradient Descent



Mini-Batch Gradient Descent



Stochastic Gradient Descent



A compromise between computing the true gradient and the gradient at a single sample is to compute the gradient against more than one training sample (called a "mini-batch") at each step.

Conjugate Gradient method

Given a differentiable function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ and an initial approximation a_0 , one starts in the steepest descent direction:

$$\Delta a_0 = -\nabla f(a_0)$$

Find the step size $\alpha_0 := \arg\min_{\alpha} f(a_0 + \alpha \Delta a_0)$ and $a_1 = a_0 + \alpha_0 \Delta a_0$.

After this iteration, the following steps with $n=1, 2, \dots$ constitute one iteration of moving along a subsequent conjugate direction s_n , where $s_0 = \Delta a_0$:

1. Calculate the steepest direction $\Delta a_n = -\nabla f(a_n)$.
2. Compute β_n according to certain formulas. (see below)
3. Update the conjugate direction $s_n = \Delta a_n + \beta_n s_{n-1}$ ($s_{n-1}^T \cdot H \cdot s_n = 0$). ←
4. Find $\alpha_n = \arg\min_{\alpha} f(a_n + \alpha s_n)$.
5. Update the position: $a_{n+1} = a_n + \alpha_n s_n$.

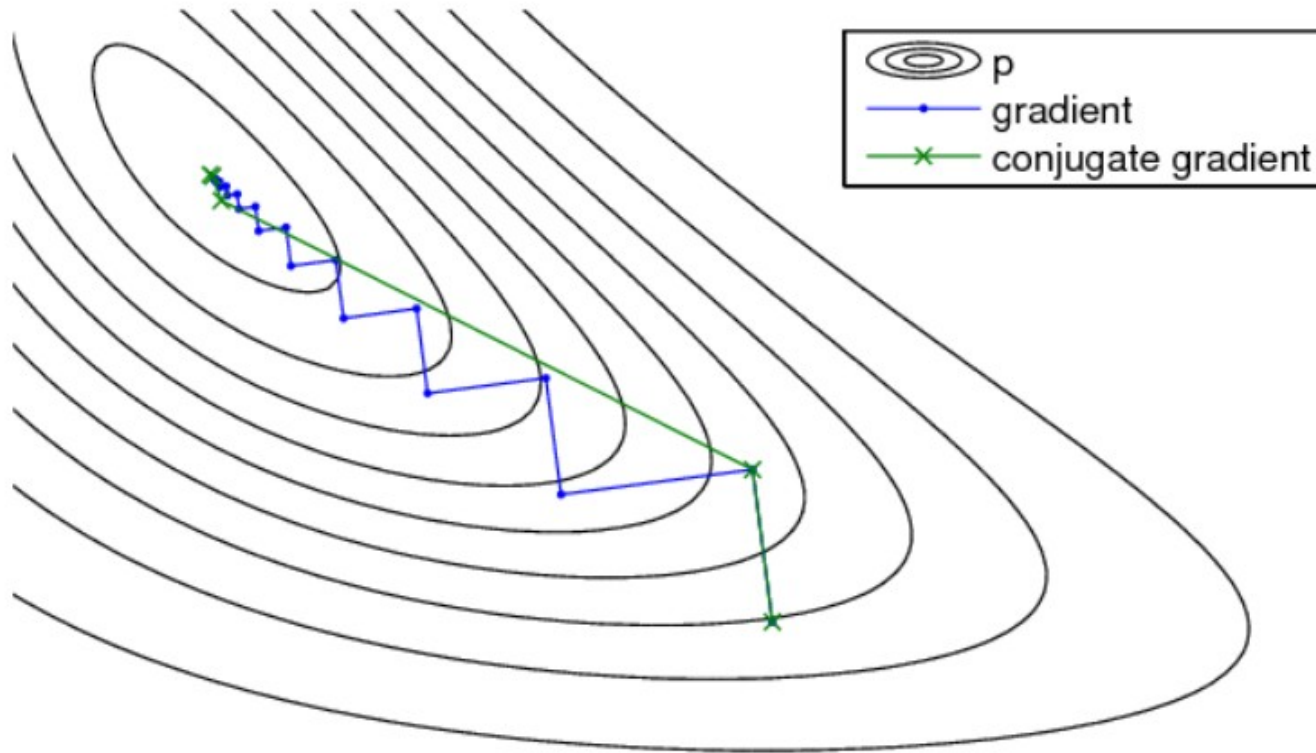
The choice of β_n (to guarantee the uniform convergence $a_n \rightarrow x^*$ as $n \rightarrow \infty$ for convex f and Lipschitz ∇f) is due to, say, Fletcher-Reeves or Polak-Ribiere: ←

$$\beta_n^{FR} = \frac{\Delta a_n^T \Delta a_n}{\Delta a_{n-1}^T \Delta a_{n-1}} \quad \beta_n^{PR} = \frac{\Delta a_n^T (\Delta a_n - \Delta a_{n-1})}{\Delta a_{n-1}^T \Delta a_{n-1}}$$

$S_1, S_2, \dots, S_n$ are conjugate
if $S_i^T \cdot H \cdot S_j = 0 \quad i \neq j$
 $S_i^T \cdot H \cdot S_i \geq 0$
 $i, j = 1, 2, 3, \dots, n$

Conjugate Gradient method

Gradient Descent oscillates much and it slows down the convergence to x^* , while the Non-linear Conjugate Gradient moves to x^* in a rather straightforward manner.



AdaGrad (Adaptive Gradient): An early adaptive method that uses the sum of squared gradients to adapt the learning rate.

RMSProp (Root Mean Square Propagation): Addresses the diminishing learning rates of AdaGrad by using an exponentially decaying average of squared gradients.

Adam (Adaptive Moment Estimation): Combines the advantages of RMSProp and AdaGrad by incorporating both momentum and adaptive learning rates.

Newton's method. One-dimensional case

Let $f: \mathbb{R} \rightarrow \mathbb{R}$ be convex and twice differentiable. Find a root of $f'=0$ by constructing a sequence a_n from an initial approximation a_0 so that $a_n \rightarrow x^*$ as $n \rightarrow \infty$, where $f'(x^*) = 0$. From the Taylor expansion of f near a_n ,

$$f(a_n + \Delta a) \approx T(\Delta a) := f(a_n) + f'(a_n)\Delta a + \frac{1}{2}f''(a_n)(\Delta a)^2$$

We use this quadratic function (with respect to Δa) as an approximant to f in a neighbourhood of a_n . The vertex of the corresponding parabola gives us the next point a_{n+1} . To find the vertex x-coordinate, we write:

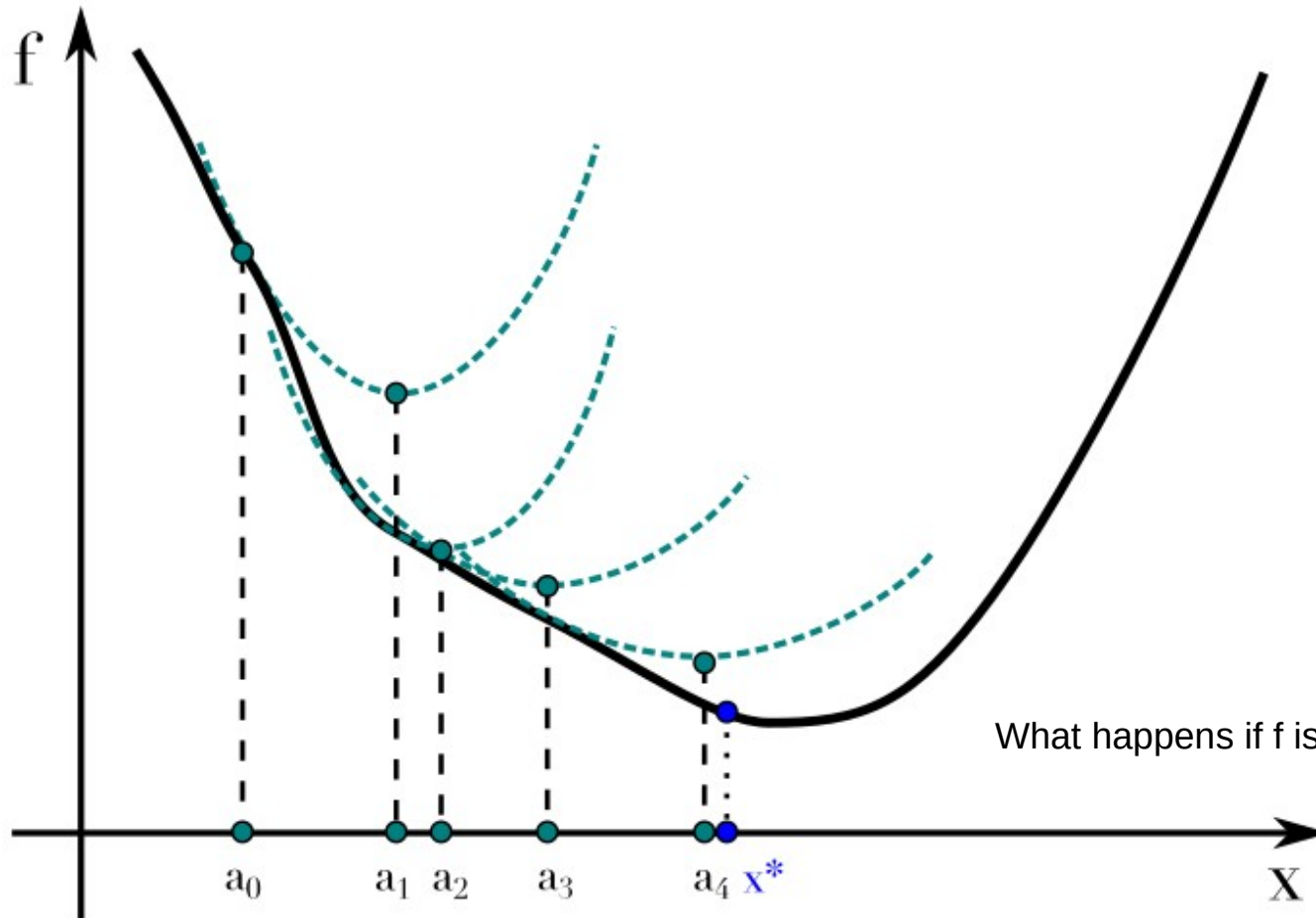
$$0 = \frac{dT_f(\Delta a)}{d\Delta a} = f'(a_n) + f''(a_n)\Delta a \Rightarrow \Delta a = -\frac{f'(a_n)}{f''(a_n)}$$

Here $f''(x) > 0$ (why?). Incrementing a_n by this Δa gives us a point closer to x^* :

$$a_{n+1} = a_n + \Delta a = a_n - \frac{f'(a_n)}{f''(a_n)}$$

It is proved that for the chosen class of f one has $a_n \rightarrow x^*$ as $n \rightarrow \infty$.

Newton's method. One-dimensional case



What happens if f is a quadratic function?

Newton's method. Multidimensional case

Let $f: \mathbb{R}^n \rightarrow \mathbb{R}$ be convex and $Hf(x)$ is invertible for $x \in \mathbb{R}^n$.

The one-dimensional scheme can be generalized to several dimensions by replacing the derivative with the gradient, ∇f , and the reciprocal of the second derivative with the inverse of the Hessian matrix:

$$a_{n+1} = a_n - [Hf(a_n)]^{-1} \nabla f(a_n), n=0,1,\dots$$

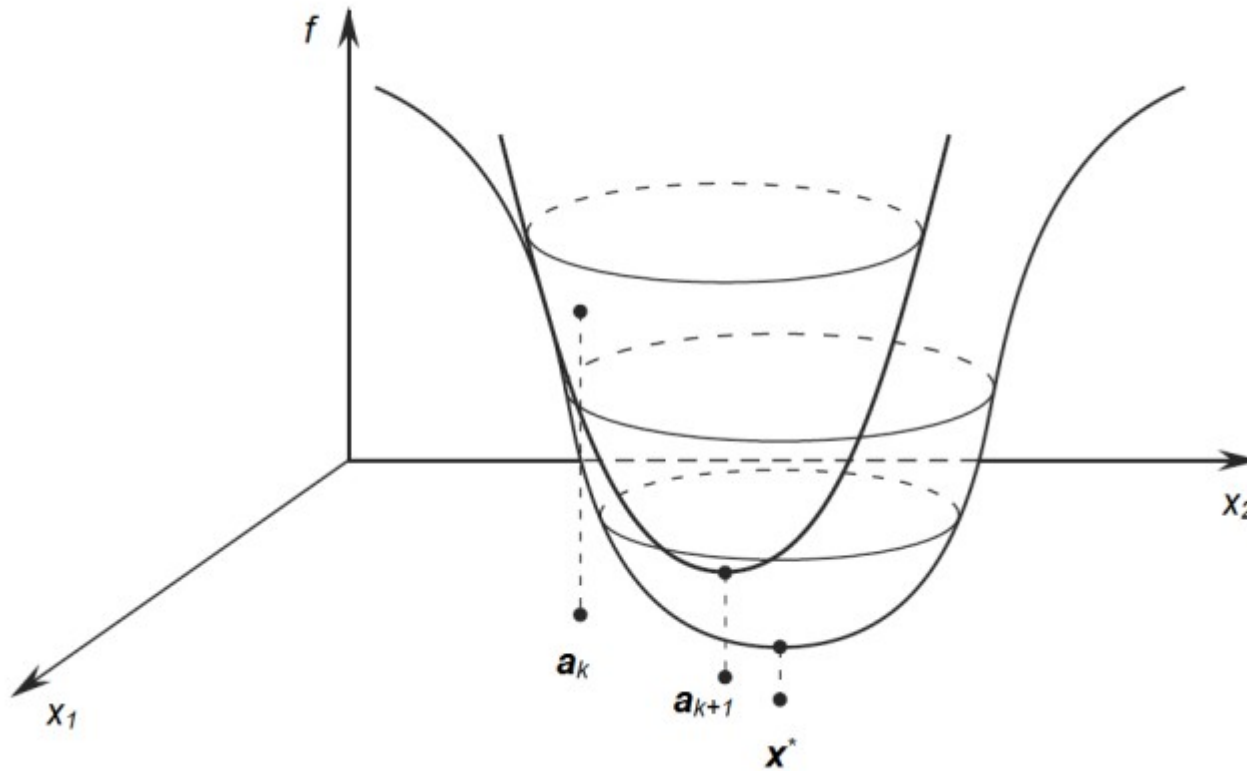
By decreasing the step size, one obtains the relaxed Newton's method:

$$a_{n+1} = a_n - \beta [Hf(a_n)]^{-1} \nabla f(a_n), \beta \in (0,1), n=0,1,\dots$$

to guarantee the method's convergence. One often supposes then that f is strictly convex and Hf is Lipschitz (satisfies Lipschitz conditions).

$$d_Y(f(x_1), f(x_2)) \leq K d_X(x_1, x_2)$$

Newton's method. Multidimensional case **VITMO**



Quasi-Newton method in ML

Hessian-free methods using the conjugate gradient algorithm to solve sub-optimization problems **without explicitly forming the Hessian matrix**.

Gauss-Newton methods

Approximate the Hessian using the Gauss-Newton matrix

Limited-Memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS)

One of the most popular quasi-Newton methods used in ML. It constructs positive definite Hessian approximations using only gradient information and is particularly useful for large-scale problems where storing the full Hessian matrix is impractical. L-BFGS methods have been adapted for stochastic settings and have shown robust convergence and good generalization properties in deep learning applications.

Broyden-Fletcher-Goldfarb-Shanno (BFGS)

A fundamental quasi-Newton algorithm that updates the Hessian approximation at each iteration using the quasi-Newton condition. It is known for its "self-correcting" qualities and is highly effective. However, its full-memory version can be computationally expensive for large-scale problems.

Stochastic Quasi-Newton Methods

These methods have been developed to handle the challenges of large-scale ML problems. Stochastic L-BFGS with variance reduction combines the efficiency of L-BFGS with the robustness of variance reduction techniques.

Kronecker-Factored Block-Diagonal BFGS and L-BFGS

These methods are specifically designed for training deep neural networks. They use Kronecker-factored block-diagonal approximations to the Hessian matrix, which are more efficient and scalable for large neural networks.

Levenberg-Marquardt algorithm (LMA)

The LMA is a pseudo-second order method, its application is to solve non-linear least squares problems.

For a set $(x_i, y_i)_{i=1}^m$, find a column-vector β of the parameters $\beta = (\beta_1 \cdots \beta_n)$ of the function $f(x, \beta)$ such that

$$\beta = \underset{\beta}{\operatorname{argmin}} S(\beta); S(\beta) = \sum_{i=1}^m [y_i - f(x_i, \beta)]^2$$

Start with an initial guess for β . At each iteration, β is replaced $\beta + \Delta \beta$.

To determine $\Delta \beta$, $f(x_i, \beta + \Delta \beta)$ is approximated by its linearization:

$$f(x_i, \beta + \Delta \beta) \approx f(x_i, \beta) + J_i(\Delta \beta); J_i = \left(\frac{\partial f(x_i, \beta_j)}{\partial \beta_j} \right)_{j=1}^n$$

$$(H(x) = 2J^T(x)J(x) + 2 \sum_{i=1}^m f_i(x) H_i(x))$$

The sum $S(\beta)$ has its minimum at a zero gradient with respect to β .

The above-mentioned linear approximation of

$f(x_i, \beta + \Delta \beta)$ gives

$$S(\beta + \Delta \beta) \approx \sum_{i=1}^m [y_i - f(x_i, \beta) - J_i \Delta \beta]^2$$

in vector notation: $S(\beta + \Delta \beta) \approx [y - f(\beta)]^T [y - f(\beta)] - 2[y - f(\beta)]^T J \Delta \beta + \Delta \beta^T J^T J \Delta \beta$

where J is the Jacobian matrix, whose i -th row equals J_i , and where $f(\beta)$, y , β are column-vectors with i -th component $f(x_i, \beta)$, y_i , β_i , respectively.

Taking the derivative of $S(\beta + \Delta \beta)$ with respect to $\Delta \beta$ and setting to zero gives $J^T J \Delta \beta = J^T [y - f(\beta)]$

Levenberg-Marquardt algorithm (LMA)

Taking the derivative of $S(\beta + \Delta\beta)$ with respect to $\Delta\beta$ and setting to zero gives

$$J^T J \Delta\beta = J^T [y - f(\beta)]$$

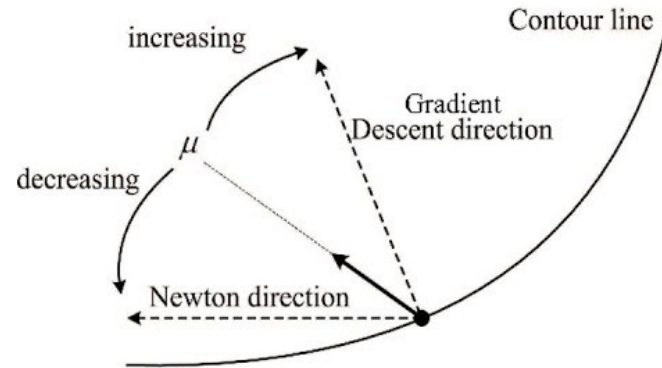
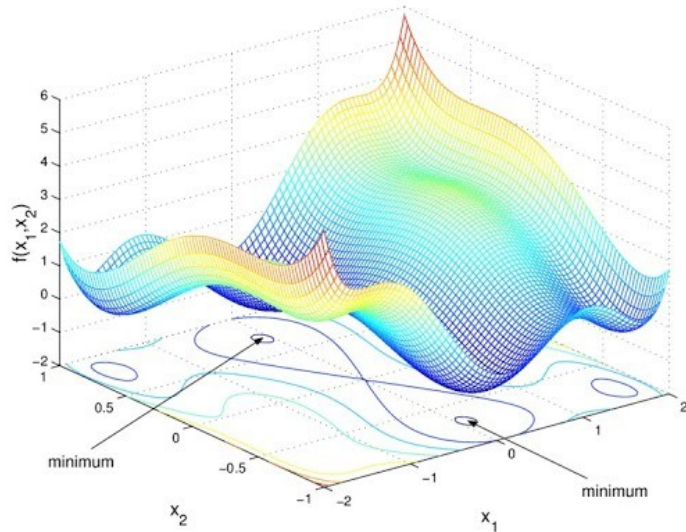
that is in fact a system of linear equations with respect to $\Delta\beta$.

The system may be replaced by the following damped version:

$$(J^T J + \lambda I) \Delta\beta = J^T [y - f(\beta)]$$

where I is the identity matrix, giving the increment $\Delta\beta$ to the estimated parameter vector β

The damping factor $\lambda > 0$ is adjusted at each iteration and should be chosen to guarantee the method's convergence.



Levenberg-Marquardt algorithm (LMA)

The Levenberg-Marquardt Algorithm (LMA) is considered to interpolate between the Gauss-Newton algorithm and the Gradient Descent method due to the way it adjusts the damping parameter λ , which influences the update step in the optimization process.

Gauss-Newton Component

The Gauss-Newton algorithm is used to solve non-linear least squares problems by iteratively updating the parameters using the normal equations, which involve the Jacobian matrix of the residuals. The update step in the Gauss-Newton method is given by:

$$\beta^{(s+1)} = \beta^{(s)} + (J^T J)^{-1} J^T r(\beta^{(s)})$$

In the LMA, when the damping parameter λ is small, the update step approaches the Gauss-Newton update, because the term λ becomes negligible compared to $J^T J$.

Gradient Descent Component

When the damping parameter λ is large, the LMA update step resembles that of Gradient Descent.

$$(J^T J + \lambda I) \Delta \beta = J^T [y - f(\beta)]$$

If λ is large relative to $J^T J$, the term λ dominates, and the update is approximated by a gradient descent step. The LMA adaptively adjusts the value of λ based on the success of the previous iteration. This is essentially a step in the direction of the negative gradient of the sum of squares, which is characteristic of Gradient Descent.



АНТОН КОВАНЦЕВ
pop.antonij@gmail.com
https://t.me/pop_antonij